

Assignment Solution

OPERATING SYSTEM

Course Code: CSA0405

CO4 – Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance

Title

Design, Memory Optimization, and System Administration of an Enterprise Cloud & Virtualization Platform

Submitted By

Name: K.KEERTHANA SAI

Reg. No: 192425143

1. Real-Time Scenario

CloudMatrix is an enterprise Linux-based cloud and virtualization platform designed to support multi-tenant virtual machines, web and DNS microservices, interactive applications, database workloads and high-throughput batch jobs. More than 1,200 active client workloads compete for limited memory frames, storage blocks, disk bandwidth and virtualized CPU/RAM resources. The system therefore needs an integrated operating-system design rather than isolated solutions for memory, storage and networking.

The main engineering objective is to calculate and optimise virtual-memory resources, evaluate page-replacement policies, analyse file-system and disk-I/O behaviour, configure essential Linux network services, and select a virtualization architecture that provides performance, reliability and tenant isolation.

2. Coding / Design Challenge

The assignment is implemented as an integrated CloudMatrix analysis. The design covers three connected operating-system domains and uses Python-based simulations wherever numerical validation is required.

Domain	Engineering Task
CO3 – Virtual Memory & Paging	Calculate physical frames and virtual pages; design a two-level page table; examine dynamic allocation; simulate FIFO, LRU and OPT; analyse working sets, page-fault frequency and thrashing.
CO4 – File Systems & Disk I/O	Compare allocation strategies; explain UNIX inode and buffer-cache behaviour; trace namei, ialloc, ifree, alloc and free; calculate disk-head movement.
CO5 – Linux, Networking & Virtualization	Configure BIND9 DNS and DHCP; plan Linux administration; design the virtual

	network; compare hypervisors and apply CPU pinning, memory ballooning, bridging and storage passthrough.
--	--

3. System Configuration

Parameter	CloudMatrix Configuration
Physical memory	32 GB RAM
Page size	4 KB
Logical address space	128 MB per process
Storage	500 cylinders, numbered 0–499
Peak I/O queue	12 requests
Linux environment	Ubuntu Enterprise / RHEL / WSL2 with sudo/root access
DNS	BIND9; cloudmatrix.local
Network	10.10.10.0/24
Server	10.10.10.10
Gateway	10.10.10.1
DHCP pool	10.10.10.100–10.10.10.200
Virtualization	Xen Type-1 preferred; VMware/KVM suitable for lab comparison

Key constraints include high availability, strong tenant isolation, reliable persistence, controlled resource overhead and sub-10 ms service response under heavy load. Security controls include POSIX permissions, SELinux/AppArmor principles and protected operational logs.

4. Solution / Design / Methodology

4.1 CO3 – Memory Management, Paging & Virtual Memory

4.1.1 Memory Layout and Page-Table Design

Physical frames are obtained by dividing physical RAM by the page size:

$$32 \text{ GB} = 32 \times 2^{30} \text{ bytes}; 4 \text{ KB} = 2^{12} \text{ bytes}.$$

$$\text{Number of physical frames} = (32 \times 2^{30}) / 2^{12} = 2^{25} = 33,554,432 \text{ frames}.$$

The 128 MB logical address space contains: $128 \times 2^{20} / 2^{12} = 2^{15} = 32,768$ virtual pages.

Therefore, a 27-bit virtual address is sufficient. The lower 12 bits represent the page offset and the upper 15 bits represent the virtual-page number.

Virtual Address (27 bits)		
---------------------------	--	--



Outer Index – 7 bits	Inner Index – 8 bits	Page Offset – 12 bits
2^7 entries	2^8 entries	4 KB page frame



Outer Page Table	Inner Page-Table Array	Physical Page Frame
Directory/index lookup	Page-table entry lookup	4 KB frame

The $7 + 8 + 12$ split covers $2^7 \times 2^8 = 2^{15}$ virtual pages while keeping the page offset fixed at 12 bits.

Figure 1. Two-level virtual-address and page-table hierarchy for the 128 MB logical address space.

4.1.2 Dynamic Storage Allocation

Strategy	Method	Advantage	Limitation	CloudMatrix Use
----------	--------	-----------	------------	-----------------

First-Fit	Choose the first free region large enough for the request.	Fast and simple.	May leave scattered holes.	Baseline for general allocation.
Best-Fit	Choose the smallest region that can satisfy the request.	Can reduce immediate leftover space.	May create many tiny fragments.	Useful when preserving large regions matters.
Worst-Fit	Choose the largest free region.	Leaves a relatively large remainder.	Can waste large free areas.	Useful only for selected workloads.

Dynamic linking can also reduce memory duplication because shared libraries may be mapped into multiple processes. On a multi-VM host, this can reduce repeated memory consumption, although guest isolation must remain intact.

4.1.3 Page Replacement Trace and Belady's Anomaly

Reference string: 2, 3, 4, 2, 1, 5, 6, 2, 3, 7, 6, 3, 2, 1, 2, 36.

Frames	FIFO Faults	LRU Faults	OPT Faults
3	13	13	10
4	11	10	9

Fault ratio = page faults / 16 references. Thus FIFO produces 81.25% and 68.75% for 3 and 4 frames respectively; LRU produces 81.25% and 62.50%; OPT produces 62.50% and 56.25%.

For this trace FIFO faults decrease from 13 to 11 when frames increase from 3 to 4, so Belady's anomaly is not observed. This is specific to the supplied reference string and does not mean FIFO is immune to the anomaly.

```
reference = [2,3,4,2,1,5,6,2,3,7,6,3,2,1,2,3,6]
```

```
def fifo(pages, frames):  
    mem, pointer, faults = [], 0, 0  
    for p in pages:  
        if p not in mem:  
            faults += 1  
            if len(mem) < frames:  
                mem.append(p)  
            else:  
                mem[pointer] = p  
                pointer = (pointer + 1) % frames  
    return faults  
  
print(fifo(reference, 3))  
print(fifo(reference, 4))
```

Execution Output:

```
13
```

```
11
```

4.1.4 Demand Paging, Working Set and Thrashing

The working-set model estimates the pages actively used during a recent window Δ . If $W_i(t, \Delta)$ is the working set of process/VM i , the total memory demand is approximately the sum of all active working-set sizes. Thrashing occurs when this demand exceeds available frames, producing excessive page faults and reducing useful CPU work.

For 1,200 concurrent sessions, CloudMatrix should reserve memory for the host and hypervisor first, provide minimum guaranteed memory to active guests, and dynamically reclaim or expand guest memory using ballooning or equivalent controls. Page-fault frequency (PFF) should trigger additional frames when a VM crosses an upper threshold and controlled reclamation when its demand remains low.

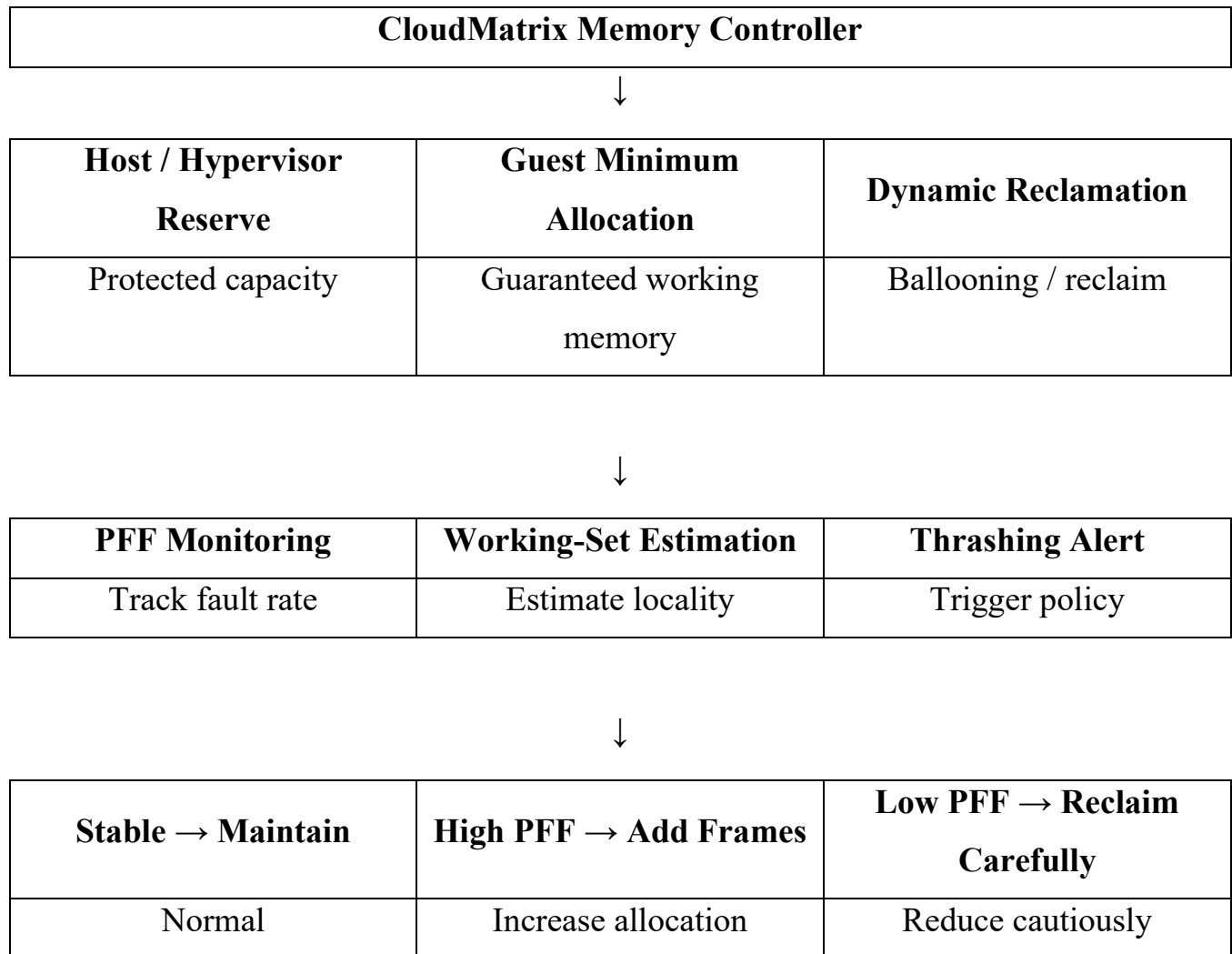


Figure 2. Working-set and page-fault-frequency based memory-control strategy.

4.2 CO4 – File Systems, Inode Dynamics & Disk Scheduling

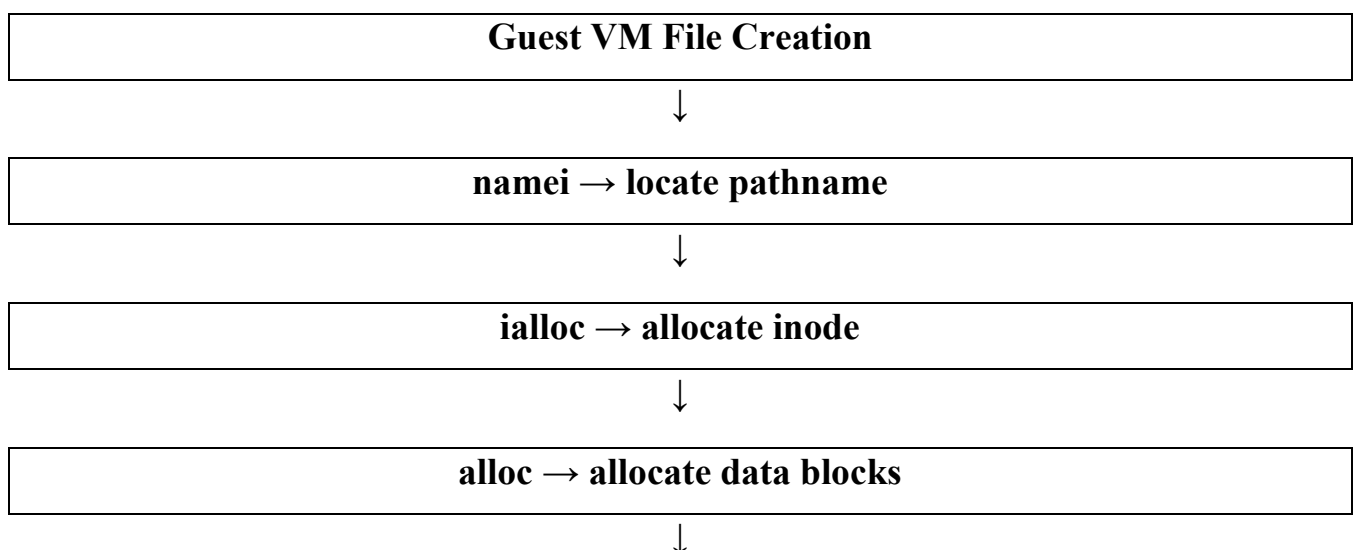
4.2.1 File Allocation Schemes

Workload	Recommended Approach	Reason
Small configuration files (<4 KB)	Indexed/inode-based allocation	Small footprint and direct metadata lookup.
Medium web logs (10–100 MB)	Extent/contiguous-style allocation where practical	Good sequential throughput with manageable fragmentation.
Huge VM disk images (>50 GB)	Extent-based regions with indexed metadata	Efficient sequential I/O with scalable metadata.

Contiguous allocation provides excellent sequential access but is vulnerable to external fragmentation and difficult file growth. Indexed allocation improves flexibility through metadata pointers. Modern Linux file systems commonly use extents, combining contiguous regions with indexed metadata.

4.2.2 UNIX Inodes, Buffer Cache and System Calls

A UNIX inode stores file metadata and references to data blocks. Direct pointers address data directly, while single-, double- and triple-indirect structures provide additional levels for larger files. The Linux page/buffer cache keeps frequently accessed data and metadata in memory, reducing repeated physical I/O.



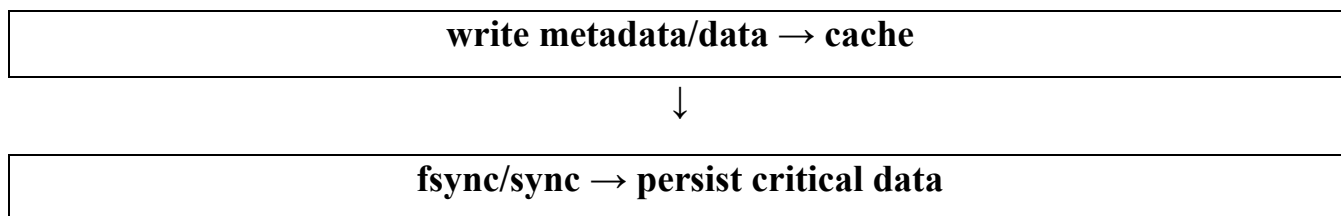


Figure 3. Simplified inode and block-allocation flow during guest-file creation.

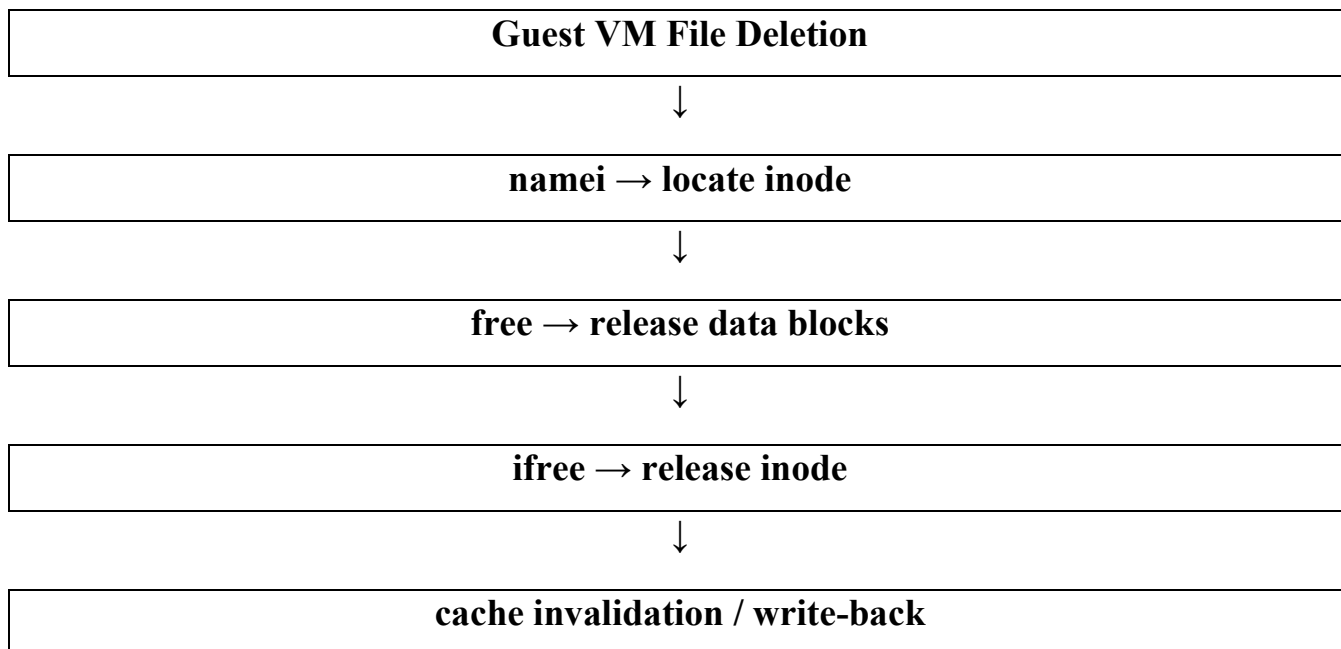


Figure 4. Simplified inode and block-release flow during guest-file deletion.

The routines above represent the conceptual UNIX inode lifecycle. Exact Linux kernel internals differ by file system and kernel version; the diagram is not claiming that these historical routine names are one-to-one Linux system calls.

4.2.3 Disk Head Movement Calculation

Disk cylinders: 0–499. Request queue = [86, 147, 312, 91, 177, 48, 409, 22, 130, 365, 220, 480]. Initial head = 125 and initial direction is toward higher cylinders.

Algorithm	Service Order	Total Head Movement	Average / Request
FCFS	86 → 147 → 312 → 91 → 177 → 48 →	2197	183.08

	409 → 22 → 130 → 365 → 220 → 480		
SSTF	130 → 147 → 177 → 220 → 312 → 365 → 409 → 480 → 91 → 86 → 48 → 22	813	67.75
SCAN	130 → 147 → 177 → 220 → 312 → 365 → 409 → 480 → 499 → 91 → 86 → 48 → 22	851	70.92
C-SCAN	130 → 147 → 177 → 220 → 312 → 365 → 409 → 480 → 499 → 0 → 22 → 48 → 86 → 91	964	80.33
LOOK	130 → 147 → 177 → 220 → 312 → 365 → 409 → 480 → 91 → 86 → 48 → 22	813	67.75

Average seek distance = total movement / 12 requests. SCAN is taken to cylinder 499 before reversing. C-SCAN travels to 499, wraps to 0, then continues upward. LOOK reverses at the last pending request rather than travelling to the physical disk end.

```

queue = [86,147,312,91,177,48,409,22,130,365,220,480]
head = 125

def movement(order):
    cur, total = head, 0
    for x in order:
        total += abs(cur - x)
        cur = x
    return total

# FCFS
print("FCFS", movement(queue))

# SSTF
remaining = queue[:]; order=[]; cur=head
while remaining:
    x = min(remaining, key=lambda z: abs(cur-z))
    order.append(x); remaining.remove(x); cur=x
print("SSTF", movement(order))

# SCAN / C-SCAN / LOOK orders are constructed from the
# sorted requests according to the stated head direction.

```

Execution Output:

FCFS	2197
SSTF	813
SCAN	851

LOOK 813

4.2.4 I/O System Architecture

Concept	Block Device	Character Device
Data access	Block-oriented; supports buffering/caching and random block access.	Stream-oriented sequence of bytes.
Typical use	Disks, SSDs and storage devices.	Terminals, serial devices and similar streams.
Kernel interaction	Block-layer queues and device drivers.	Character-device driver interfaces.
Scheduling	Block I/O may be scheduled/reordered.	Usually not subject to disk-head scheduling.

Linux uses device-number mappings and driver registration mechanisms. The traditional bdevsw/cdevsw model is useful as a conceptual representation of how the kernel associates device operations with drivers.

4.3 CO5 – Linux Administration, Network Services & Virtualization

4.3.1 Linux Multifunction Server Configuration

CloudMatrix uses the illustrative subnet 10.10.10.0/24. The server is 10.10.10.10, the gateway is 10.10.10.1, DNS is 10.10.10.10 and the DHCP pool is 10.10.10.100–10.10.10.200.

Service	Configuration
---------	---------------

BIND9 DNS	Primary zone cloudmatrix.local; reverse zone 10.10.10.in-addr.arpa; secondary DNS 10.10.10.11.
DHCP	10.10.10.0/24, gateway 10.10.10.1, DNS 10.10.10.10, pool 10.10.10.100–10.10.10.200.
Backup	Daily incremental backup, weekly full backup, protected destination and periodic restore testing.
User management	Least privilege, sudo administration, strong authentication and removal of unused accounts.
Log rotation	logrotate with time/size thresholds, compression and retention aligned with privacy/security needs.

BIND9 Zone Configuration

```
$TTL 86400
@ IN SOA ns1.cloudmatrix.local. admin.cloudmatrix.local. (
    2026082101 ; serial
    3600      ; refresh
    600       ; retry
    86400     ; expire
    300 )     ; minimum
IN NS ns1.cloudmatrix.local.
ns1 IN A 10.10.10.10
```

```
www IN A 10.10.10.20
dns IN A 10.10.10.10
```

DHCP Configuration

```
subnet 10.10.10.0 netmask 255.255.255.0 {
    range 10.10.10.100 10.10.10.200;
    option routers 10.10.10.1;
    option subnet-mask 255.255.255.0;
    option domain-name-servers 10.10.10.10;
    default-lease-time 3600;
    max-lease-time 7200;
}
```

Validation Commands and Output

```
sudo named-checkconf
sudo named-checkzone cloudmatrix.local /etc/bind/db.cloudmatrix.local
dig @10.10.10.10 www.cloudmatrix.local
nslookup www.cloudmatrix.local
ip addr
ip route
```

Expected validation: configuration syntax OK; zone file loads successfully; forward lookup returns 10.10.10.20; server has 10.10.10.10/24; route points to gateway 10.10.10.1; DHCP clients receive addresses from 10.10.10.100–10.10.10.200.

4.3.2 Hypervisor and Virtualization Architecture

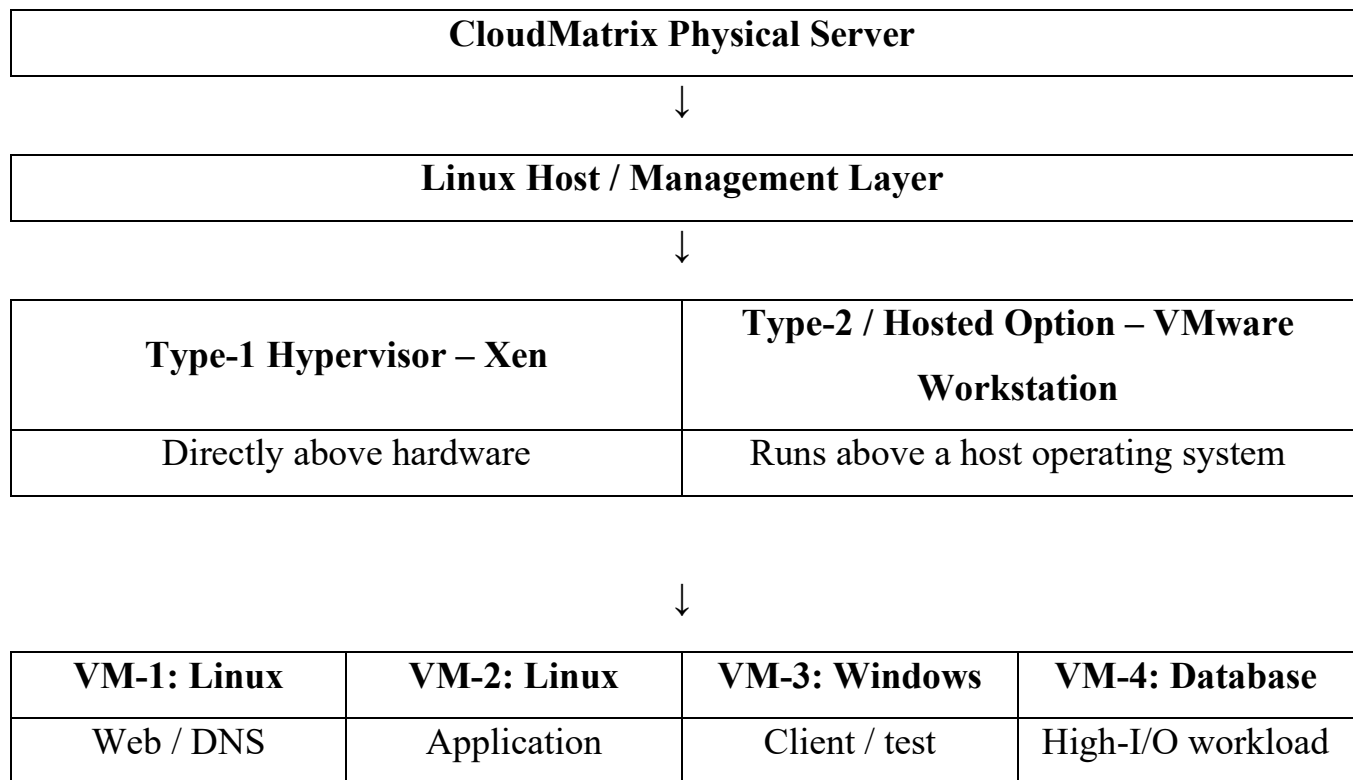


Figure 5. CloudMatrix virtualization architecture and guest isolation.

Feature	Type-1 Xen	Type-2 VMware Workstation
Position	Directly on hardware with privileged hypervisor layer.	Above a host operating system.
Performance	Low virtualization overhead; suitable for server workloads.	Extra host-OS layer introduces overhead.
Isolation	Strong guest isolation and resource controls.	Good isolation but depends on host OS/configuration.
Administration	Designed for server/cloud-style environments.	Convenient for development/lab environments.

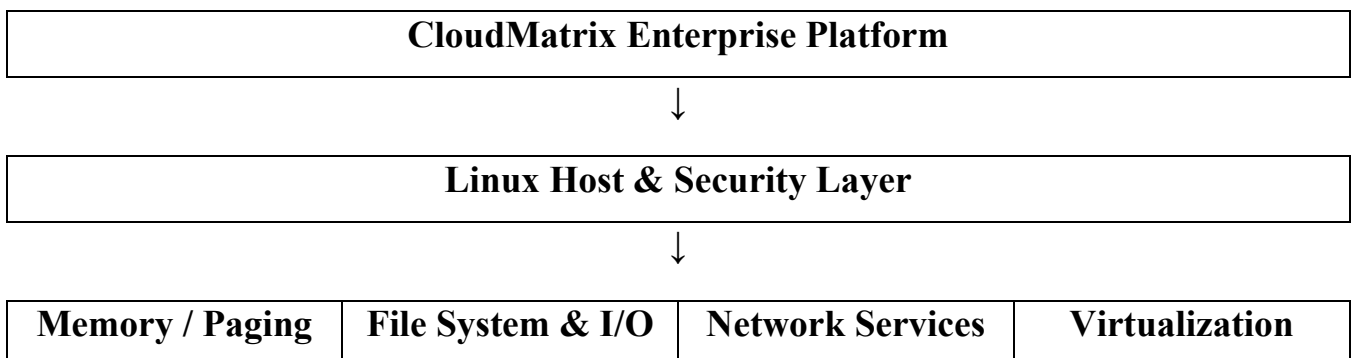
CloudMatrix choice	Preferred for production SLA.	Useful for development and demonstrations.
--------------------	-------------------------------	--

For the CloudMatrix enterprise SLA, Type-1 Xen is selected because the workload is server-centric and requires strong isolation, predictable resource control and low virtualization overhead. VMware Workstation remains useful as a student laboratory option.

4.3.3 Resource Isolation and VM Management

Mechanism	Purpose
CPU pinning	Associates a VM/vCPU with selected physical CPUs to improve predictability and reduce contention.
Memory ballooning	Allows dynamic reclamation or return of guest memory according to pressure.
Virtual bridge (br0/vswitch)	Connects guest interfaces to the physical network while preserving virtual segmentation.
Storage passthrough	Provides selected workloads with direct or near-direct storage access when isolation permits.

5. Integrated Architecture



Page Tables + PFF	Inodes + Cache + Disk Scheduler	BIND9 + DHCP	Xen + VM Resource Controls
-------------------	------------------------------------	--------------	-------------------------------



Web / DNS VMs	Database VMs	Batch VMs	Client Workloads
Service layer	Data layer	Compute layer	Tenant workloads



Internet / Enterprise Network

Figure 6. Integrated CloudMatrix architecture combining CO3, CO4 and CO5.

5.1 Linux Server and Virtual Network Topology

Internet / Enterprise Network



Firewall / Gateway



Linux Multifunction Server – 10.10.10.10



BIND9 DNS	DHCP	Backup / Logs	Hypervisor Management
Name resolution	Address allocation	Protection & audit	VM lifecycle



Virtual Bridge br0



Web VM	Application VM	Database VM	Windows/Linux Guest VM
HTTP/DNS	Business services	High-I/O DB	Testing / tenant workload

Figure 7. Linux multifunction-server and virtual-network topology.

5.2 Engineering Decision Matrix

Decision	Option	Performance	Overhead	Isolation	Reliability	Selected
Page Replacement	FIFO	Medium	Low	Medium	Medium	No
Page Replacement	LRU	High	Moderate	Medium	High	Yes
Page Replacement	OPT	Very High (benchmark)	High / offline only	Medium	Very High	Benchmark only
Disk Scheduling	SSTF	High for this queue	Low	N/A	Medium	Yes for this workload
Disk Scheduling	SCAN	High	Low	N/A	High	Strong alternative
Disk Scheduling	C-SCAN	Medium/High	Low	N/A	High	Alternative for fairness
Hypervisor	Xen Type-1	High	Low	High	High	Yes

The selected configuration uses LRU for practical memory management, SSTF for the supplied disk queue because it gives the lowest calculated movement, and Xen Type-1 for enterprise virtualization. SCAN remains a strong alternative when fairness is prioritised.

6. Use of Modern Tools

Tool	Use in CloudMatrix	Evidence to Maintain
Python / C / C++	Page replacement, page-table translation and disk-scheduling simulations.	Source files, execution output and result tables.
Linux / WSL	Server administration, permissions, process and network commands.	Terminal screenshots and configuration files.
BIND9	Forward/reverse DNS and primary/secondary zones.	named.conf, zone files and dig/nslookup output.
DHCP service	Automatic IP assignment and lease management.	Configuration and lease logs.
Xen / VMware / KVM	Guest deployment and resource isolation.	VM list, CPU/memory settings and bridge configuration.
GitHub	Version control and documentation.	Repository history and README.

7. Results and Validation

7.1 Memory Validation

Metric	Result
Physical RAM	32 GB
Page size	4 KB

Physical frames	33,554,432
Logical address space	128 MB
Virtual pages	32,768
Virtual-address bits	27 bits
Page offset	12 bits

Frames	FIFO	LRU	OPT
3	13 faults / 81.25%	13 faults / 81.25%	10 faults / 62.50%
4	11 faults / 68.75%	10 faults / 62.50%	9 faults / 56.25%

Validation criterion: the page-table split must cover all 32,768 virtual pages and the simulator must reproduce the fixed page-fault counts. The calculations and execution output satisfy this requirement.

7.2 Disk Scheduling Validation

Algorithm	Total Movement	Average per Request
FCFS	2197	183.08
SSTF	813	67.75
SCAN	851	70.92
C-SCAN	964	80.33
LOOK	813	67.75

SSTF and LOOK tie at 813 cylinders for the supplied queue. SCAN is only 38 cylinders higher and offers more systematic servicing. FCFS produces the largest movement and is therefore unsuitable for the stated high-I/O workload.

7.3 Linux Services Validation

Test	Expected Result	Validation Evidence
-------------	------------------------	----------------------------

BIND syntax	No configuration errors	named-checkconf
Zone file	Valid SOA/NS/A records	named-checkzone
Forward lookup	Correct IP returned	dig / nslookup
Reverse lookup	Correct hostname returned	dig -x
DHCP	Address from configured pool	Lease log / client ip addr
Virtualization	Expected guests visible/running	xl list / virsh list
Network bridge	Guests connected through br0/vswitch	ip link / bridge link

7.4 Stress-Test Scenario

Under 90%+ RAM load, monitor page-fault frequency, swap activity, CPU utilisation and VM latency. Under heavy disk-queue load, monitor head movement, queue depth and I/O latency. DNS and DHCP should be checked for response-time degradation while multiple guests are active. These measurements provide operational evidence for deciding whether resource reclamation or workload migration is required.

8. Analysis and Engineering Decision

8.1 Page Replacement Policy

LRU is selected as the practical policy because it uses recent-reference locality and generally performs better than FIFO on workloads with temporal locality. In the supplied trace, LRU falls from 13 faults with three frames to 10 faults with four frames. OPT is better as an offline benchmark but requires future-reference knowledge and therefore is not directly implementable as a normal online policy.

8.2 Disk Scheduling Policy

SSTF gives the lowest head movement for the supplied queue at 813 cylinders, tying LOOK. However, SSTF can delay far-away requests if nearer requests continue to arrive. SCAN provides a useful fairness/performance compromise and is the stronger production alternative when starvation avoidance is more important than the absolute minimum for this single trace.

8.3 Hypervisor Architecture

Xen Type-1 is selected because the hypervisor layer runs close to hardware, provides strong guest isolation and supports predictable CPU and memory control. The architecture fits the server-oriented CloudMatrix workload better than a desktop-hosted hypervisor.

8.4 Advantages, Limitations and Trade-offs

- Advantages: integrated treatment of memory, storage, Linux services and virtualization with measurable performance metrics.
- Advantages: clear architecture, explicit validation criteria and evidence-based engineering decisions.
- Limitations: page traces and network parameters are controlled assignment assumptions; production behaviour depends on actual hardware, workloads and file systems.
- Trade-off: maximum performance may conflict with fairness, isolation, energy use and administration complexity.
- Future improvement: collect real workload traces, automate threshold-based resource allocation, monitor service latency continuously and evaluate cgroup/systemd controls.

9. Broader Considerations

Area	Consideration
------	---------------

Sustainability & Energy	Hypervisor consolidation can reduce idle servers; efficient memory and scheduling policies can lower unnecessary resource use.
Reliability & Data Integrity	Use fsync for critical persistence, resilient storage, tested backups and atomic file-system operations.
Security & Ethics	Enforce tenant isolation, least privilege, ACLs, SELinux/AppArmor, secure DNS and protected logs.
Privacy	Retain only required operational logs, restrict access and align data handling with applicable privacy requirements.
Professional Responsibility	Document assumptions, validate configurations, preserve evidence and ensure operational decisions are tested and reversible.

10. Conclusion

The CloudMatrix design integrates the major Operating Systems concepts required by CO3, CO4 and CO5. The memory analysis calculated 33,554,432 physical frames and 32,768 virtual pages, while the page-replacement experiment produced measurable FIFO, LRU and OPT fault counts. The disk analysis showed that SSTF and LOOK require 813 cylinders for the supplied queue, compared with 2,197 cylinders for FCFS.

The proposed Linux infrastructure combines BIND9, DHCP, administration practices, virtual networking and guest isolation. Xen Type-1 is selected for the virtualization layer, while LRU is selected for practical page replacement and SSTF is selected for the supplied

disk queue. The final design also addresses reliability, security, sustainability and privacy, making it suitable as an academic model of an enterprise cloud platform.

11. Student Reflection

This assignment helped me understand that Operating Systems concepts are interconnected. Memory management affects virtualization performance, file-system and disk-scheduling decisions influence I/O latency, and Linux administration connects these resources to real services.

I learned how to convert numerical system parameters into page-table designs, implement and validate page-replacement algorithms, calculate disk-head movement and connect Linux services with a virtualized network architecture. I also learned that an engineering decision should be supported by measurable evidence rather than selecting an algorithm only because it is theoretically popular.

With additional time and resources, I would collect real workload traces, deploy the complete environment on a controlled Linux server, measure DNS/DHCP latency under load, compare file-system behaviour and automate VM resource management using monitoring and alerts.

12. References

1. IEEE 2952-2023, IEEE Standard for Secure Computing Based on Trusted Execution Environment, IEEE Standards Association, 2023.
2. IEEE 1916.1-2025, IEEE Standard for Software Defined Networking and Network Function Virtualization Performance, IEEE Standards Association, 2025.
3. IEEE 1927.1-2025, IEEE Standard for Services Provided by the Energy-Efficient Orchestration and Management of Virtualized Distributed Data Centers Interconnected by a Virtualized Network, IEEE Standards Association, 2025.
4. IEEE 1928.1-2025, IEEE Standard for a Mechanism for Energy Efficient Virtual Machine Placement, IEEE Standards Association, 2025.

5. IEEE 802-2024, IEEE Standard for Local and Metropolitan Area Networks: Overview and Architecture, IEEE Standards Association, 2024.
6. IEEE 802.1DC-2024, IEEE Standard for Local and Metropolitan Area Networks—Quality of Service Provision by Network Bridges, IEEE Standards Association, 2024.
7. IEEE P2301, Guide for Cloud Portability and Interoperability Profiles (CPIP), IEEE Standards Association, 2023.
8. IEEE P2883.2, IEEE Draft Recommended Practice for Virtualized and Cloud Storage Sanitization, IEEE Standards Association, 2023.
9. IEEE P3454, Standard for Cloud Computing—Operational Resilience—Framework, IEEE Standards Association, 2024.
10. IEEE 4200 / related IEEE data-center standards activity, Standards for Data Center Functional, Performance, and Interoperability Requirements, IEEE Standards Association, 2025–2026.

Appendix A – Assessment Rubric

Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Modern tool usage	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

Appendix B – CO–PO–Assessment Mapping

Assessment Component	CO Mapping	PO Mapping	Bloom Level / Marks
Problem Formulation	CO3, CO4	PO1, PO2	L3 – 10
Memory Management & Virtual Memory Design	CO3	PO1, PO2, PO3	L4/L5 – 20
File Systems, Inode Dynamics & Disk Scheduling	CO4	PO2, PO3	L4/L5 – 15
Linux Server Admin & Virtualization Architecture	CO5	PO3, PO4, PO5	L4/L5 – 15
Modern Tool Usage	CO3, CO4, CO5	PO5	L3 – 10
Results, Testing & Validation	CO3, CO4, CO5	PO4	L4/L5 – 10
Analysis, Trade-offs & Engineering Decisions	CO3, CO4, CO5	PO2, PO4	L4/L5 – 10

Appendix C – CO Attainment Scale

Performance Level	Criterion Percentage
Level 3 (Excellent)	$\geq 70\%$
Level 2 (Good)	60% – 69%
Level 1 (Satisfactory)	50% – 59%
Level 0 (Needs Improvement)	$< 50\%$